

UNIVERSIDAD AUTÓNOMA DE TAMAULIPAS

Facultad de Ingeniería "Arturo Narvarro Siller"

Programación de Sistemas de Base 2

Traductor de Texto a Voz con Analizador Léxico-Sintáctico

Proyecto integrador semestral

Materia	Programación de Sistemas de Base 2
Proyecto	Evolución del proyecto EaV2 con lexer, parser, AST, pruebas y TTS
Equipo	Mireles Jiménez Josthin Damián / Chong Alvarez Angel Ivaldi / Bravo Torres Cesar Augusto
Profesor	Muñoz Quintero Dante Adolfo
Repositorio	Programacion-de-Sistemas-de-Base-1_Proyecto (actualizado)
Fecha	2026

Documento de demostración funcional y documentación técnica.

TABLA DE CONTENIDO

1. Introducción	1
2. Objetivos	1
3. Especificación del lenguaje	2
4. Arquitectura del sistema	2
5. Implementación	3
5.1 Análisis léxico	3
5.2 Parser y construcción del AST	4
5.3 Validación semántica y TTS	4
6. Demostración funcional	5
6.1 Ejecución con entrada válida	5
6.2 Errores detectados	6
7. Estructura del proyecto y entregables	7
8. Conclusiones	8
9. Referencias	8

1. Introducción

El proyecto parte del trabajo desarrollado en Programación de Sistemas de Base 1, donde el equipo definió la gramática EaV2 para reconocer oraciones declarativas, preguntas y exclamaciones. En esta versión se reorganizó y completó la solución para que pueda ejecutarse directamente en Python sin pasos manuales adicionales, incluyendo lexer propio, parser recursivo descendente, construcción de AST, validación básica, modo interactivo, generación de archivos de salida y pruebas automatizadas.

2. Objetivos

Objetivo general. Entregar una versión completa, ejecutable y documentada del traductor de texto a voz basado en la gramática EaV2.

Objetivos específicos.

- Reconocer tokens del lenguaje con ubicación exacta de línea y columna.
- Validar la estructura de preguntas, exclamaciones y declarativas mediante un parser recursivo descendente.
- Construir un Árbol de Sintaxis Abstracta en formato legible y JSON.
- Generar salidas de prueba para entradas válidas e inválidas.
- Integrar una capa de texto a voz opcional usando pyttsx3 cuando esté disponible.
- Documentar el proyecto en un reporte final listo para la carpeta docs.

3. Especificación del lenguaje

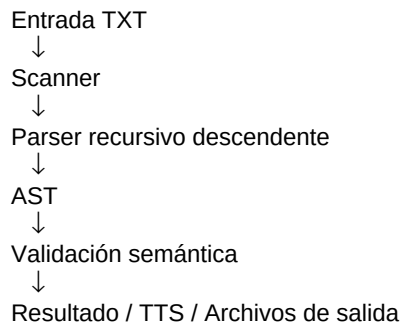
La gramática EaV2 reconoce tres tipos de sentencia: **declarative**, **question** y **exclamation**. Los términos admitidos son palabras y números; además se permiten conjunciones simples como *y*, *o* y *pero*.

Tokens reconocidos

Categoría	Ejemplos
WORD	Hola, mundo, sistema
NUMBER	3, 2025, 77
CONJ	y, o, pero
PUNCT	,,;;
LQN / RQN	¿ ?
LEXCL / REXCL	¡ !

4. Arquitectura del sistema

El flujo del sistema es secuencial: entrada de texto -> scanner -> parser -> AST -> validación semántica -> reporte de errores o envío a TTS. También se generan archivos auxiliares de salida con tokens, AST en texto, AST en JSON y resultados por cada caso de prueba.



5. Implementación

La carpeta principal del producto ejecutable es **traductor_texto_voz**. Allí se integraron los módulos necesarios para cumplir con la especificación y dejar el proyecto listo para probarse en cualquier laptop con Python.

5.1 Análisis léxico

El scanner ya no depende de archivos generados manualmente por herramientas externas para funcionar. Ahora reconoce caracteres válidos, números, signos de interrogación y exclamación, conjunciones y puntuación; además registra errores léxicos con línea y columna.

```
while i < len(text):
    if ch == '¿': token = LQN
    elif ch == '?': token = RQN
    elif ch.isdigit(): token = NUMBER
    elif palabra in {'y','o','pero'}: token = CONJ
    else: token = WORD
```

5.2 Parser y construcción del AST

Se implementó un parser recursivo descendente que sigue la forma de la gramática EaV2. A partir de la lista de tokens construye nodos para texto, sentencia, cláusula, frase, palabra y número.

5.2 Parser y construcción del AST (continuación)

sentence -> question | exclamation | declarative
question -> '¿' clause '?'
exclamation -> '¡' clause '!'
declarative -> clause PUNCT?
clause -> phrase (CONJ phrase)*
phrase -> term+

5.3 Validación semántica y TTS

Después del parser se recorre el AST para detectar frases vacías o estructuras inconsistentes. Si no existen errores, la entrada queda habilitada para el módulo de texto a voz. El motor TTS es opcional: si la librería pytsx3 no está disponible, el programa sigue funcionando y reporta la situación sin detenerse.

Manejo de errores

Se unificó el manejo de errores léxicos, sintácticos y semánticos mediante un error handler. Todos los mensajes incluyen fase, código, descripción y posición exacta.

6. Demostración funcional

6.1 Ejecución con entrada válida

Se ejecutó el archivo **examples/entrada_demo.txt**. El sistema generó tokens, AST y resultado exitoso en la carpeta **outputs**.

=== ANÁLISIS LÉXICO ===

Tokens generados: 14

=== ANÁLISIS SINTÁCTICO ===

AST construido.

=== ANÁLISIS SEMÁNTICO ===

Sin errores semánticos.

■ Entrada válida.

AST textual generado

text

sentence[1] tipo=declarative punct='.'

phrase[1] -> Hola equipo

sentence[2] tipo=question punct='?'

phrase[1] -> Como avanza el proyecto

sentence[3] tipo=exclamation punct='!'

phrase[1] -> Todo listo

6.2 Errores detectados

También se prepararon casos inválidos para demostrar el reporte funcional solicitado por el profesor.

Error sintáctico: pregunta sin signo de cierre

ERRORES Y ADVERTENCIAS

[Sintáctico] S002 - Token esperado no encontrado: Se esperaba '?' al final de la pregunta (línea 1, columna 12)

Error léxico: carácter no reconocido

ERRORES Y ADVERTENCIAS

[Léxico] L001 - Carácter desconocido: Carácter no reconocido: '@' (línea 1, columna 6)

Además, el archivo **tests/test_runner.py** ejecuta automáticamente todos los casos válidos e inválidos y confirma si el comportamiento del analizador es correcto.

7. Estructura del proyecto y entregables

La estructura final del proyecto se organizó para alinearse con lo solicitado en la especificación y en el PDF demo.

```
traductor_texto_voz/  
■■■■ grammar/      # especificación formal EaV2.g4  
■■■■ lexer/        # scanner, token, tipos  
■■■■ parser/       # parser recursivo y AST  
■■■■ semantic/     # validaciones adicionales  
■■■■ tts/          # motor de texto a voz opcional  
■■■■ tests/        # entradas válidas e inválidas + runner  
■■■■ examples/     # ejemplos rápidos  
■■■■ outputs/      # archivos de salida generados  
■■■■ docs/         # reporte final PDF  
■■■■ main.py       # punto de entrada  
■■■■ README.txt    # instrucciones rápidas
```

Entregables integrados en este ZIP

- Código fuente completo y listo para ejecución.
- Archivos de entrada de prueba válidos e inválidos en formato .txt.
- Archivos de salida generados por el traductor.
- Gramática EaV2 y artefactos históricos del proyecto original.
- Reporte PDF final dentro de docs.

8. Conclusiones

El proyecto quedó convertido en una entrega ejecutable y demostrable. La parte más importante del avance fue pasar de una base académica dispersa a una solución organizada, capaz de procesar entradas, producir salidas verificables y documentarse según el formato exigido por el profesor.

Entre las mejoras principales se encuentran: eliminación de dependencias manuales para el análisis, batería de pruebas, generación de AST en múltiples formatos, salidas listas para evidencias y reporte final estructurado.

9. Referencias

- [1] Proyecto y documentación previa del equipo sobre EaV2 y texto a voz.
- [2] Parr, T. The Definitive ANTLR 4 Reference.
- [3] Python Software Foundation. Documentación oficial de Python.
- [4] pyttsx3 documentation.